

Arquitectura de Microservicios con Contenedores

Docentes:

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Domain-Driven Design

The primary focus should be
on **domain** and domain logic

Complex domain design should
be based on a **model**

Domain-Driven Design

El Domain-Driven Design (DDD) es una metodología o enfoque de diseño de software que prioriza la lógica del negocio.

Fue concebido para manejar sistemas complejos y facilitar la conexión y la comunicación entre los aspectos técnicos y los de negocio.

El objetivo principal del DDD es alinear la implementación del software con los modelos funcionales y conceptuales del negocio, utilizando una abstracción común del dominio.

Domain-Driven Design

Los dos elementos más cruciales del DDD son el lenguaje ubicuo y los modelos de dominio.

1. El Lenguaje Ubicuo (**Ubiquity Language**): Este es un lenguaje común entre los desarrolladores y los expertos del dominio del negocio que se desarrollará en el software.
2. Los Modelos del Dominio (**Domain Models**): Son la representación conceptual de la funcionalidad específica del área de negocio

Ubiquity Language



- El objetivo principal del lenguaje ubicuo es ofrecer una forma coherente y unificada de describir los modelos y procesos de negocio, evitando confusiones y malentendidos. Este lenguaje se utiliza tanto en el código y diseño de la aplicación, como en las conversaciones diarias entre el equipo y terceros.
- Para formar el lenguaje ubicuo, es esencial que todas las partes (desarrolladores de software y expertos en el dominio) colaboren juntos en su creación. Este lenguaje debe estar basado en el dominio del problema y reflejado en el código de la aplicación.

Ubiquity Language: Ejemplo

- Vamos a considerar el desarrollo de una aplicación de chat que revisamos antes. Los elementos que formarían parte del Lenguaje Ubicuo en este caso podrían ser:
 - **Usuario:** una persona que usa la aplicación.
 - **Asistente Virtual: AI** responsable de responder las preguntas del **Usuario**
 - **Mensaje:** información que el usuario envía o recibe a/o desde el **Asistente**.
 - **Chat:** diálogo o conjunto de mensajes e interacciones entre el Usuario y el **Asistente** virtual.

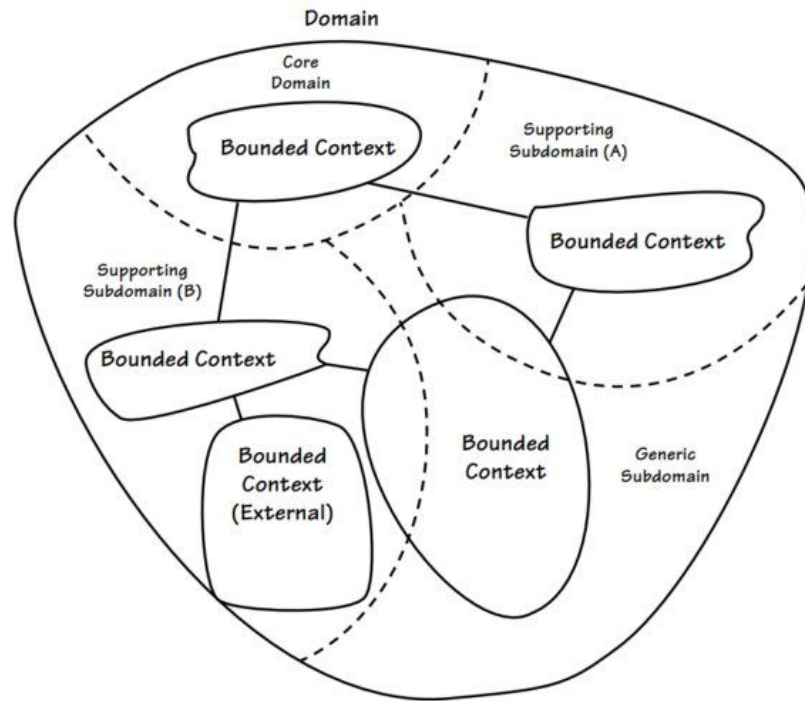


Domains

Áreas específicas de conocimiento o actividad.

No es algo escrito en código, sino más bien un mundo de ideas y conceptos.

Los subdominios, por otro lado, son como fragmentos de dominio que abarcan diferentes aspectos del problema general.



Bounded Contexts

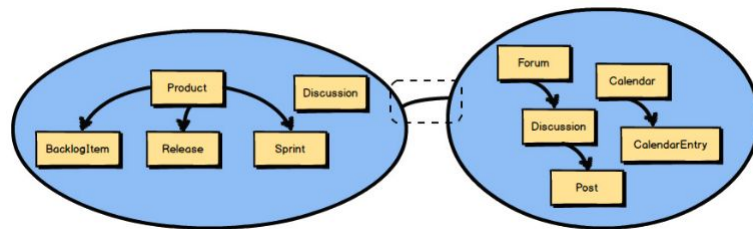
El término "Bounded Context" es un patrón de diseño estratégico que delinea un contexto limitado o delimitado dentro del cual un modelo en particular es definido y aplicado.

- Los Bounded Contexts promueven la modularidad y evitan conflictos de modelo.
- Ayudan a establecer límites claros para las reglas de negocio.
- Facilita el entendimiento y la colaboración entre equipos. Cada contexto puede ser manejado por un equipo distinto.
- Los Bounded Contexts se comunican a través de transacciones, integraciones de eventos, y otros mecanismos de intercambio de información.

Context Mapping

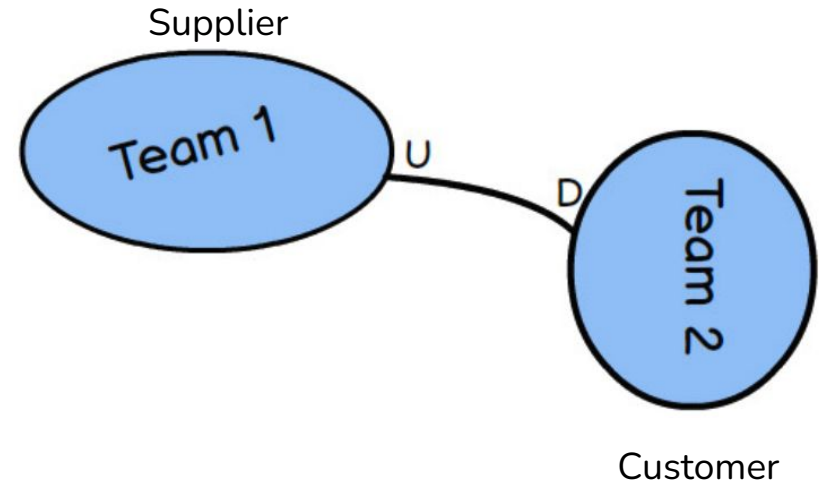
Es una herramienta que permite visualizar y entender cómo los diferentes modelos de un sistema interactúan entre sí.

- Identifica los límites de cada dominio en un sistema.
- Indica las relaciones y las interacciones entre los distintos modelos de dominio.
- Ayuda a detectar conflictos y los puntos de convergencia entre distintos contextos de dominio.



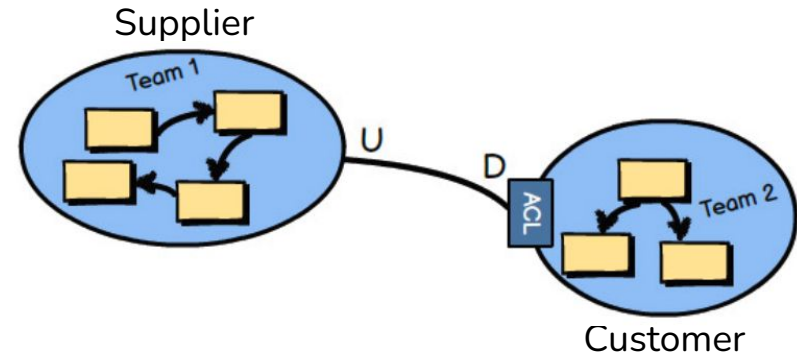
Context Mapping: Tipos de relaciones

- Customer/Supplier: Un modelo de dominio depende de la funcionalidad que otro provee.
 - Supplier es Upstream
 - Customer es Downstream
- Conformista: Un equipo debe adaptarse y seguir el modelo establecido por otro equipo.



Context Mapping: Anticorruption Layer

- Anticorruption Layer: Se establece una capa de adaptación entre dos sistemas para prevenir la corrupción del modelo propio.
- Permite la traducción del ubiquity language de un modelo a otro.

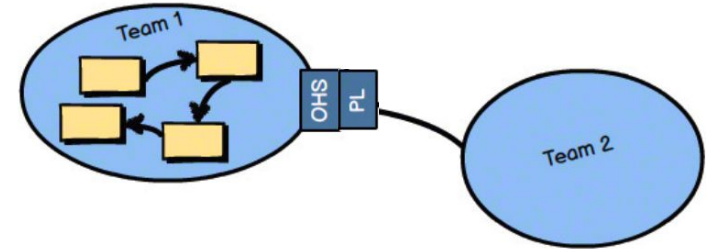


Context Mapping: Tipos de relaciones

Open Host Service: este patrón es útil cuando hay un sistema (o sub-sistema), llamado host, que debe proporcionar servicios o funcionalidades a otros sistemas.

Características:

- Bien definido: los servicios están claramente descritos y son accesibles.
- Protocolo estándar: los servicios son ofrecidos a través de una interfaz utilizando un protocolo y un **Published Language** estándar.
- Seguro y controlado: aunque el servicio es "abierto", tiene controles adecuados para garantizar que sólo los consumidores autorizados pueden acceder a él.

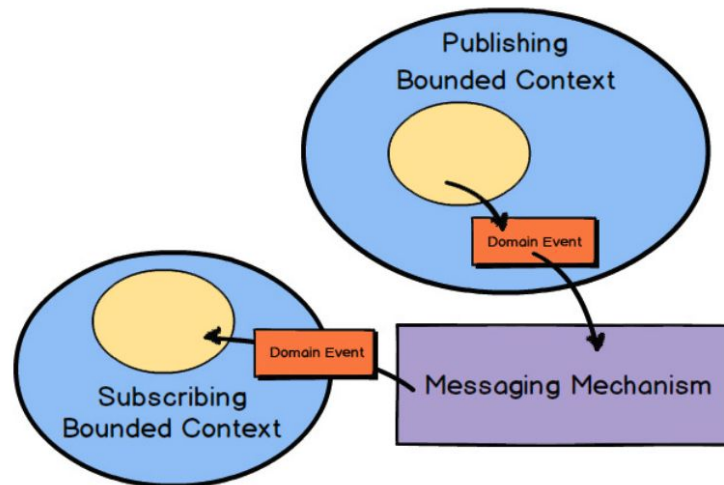
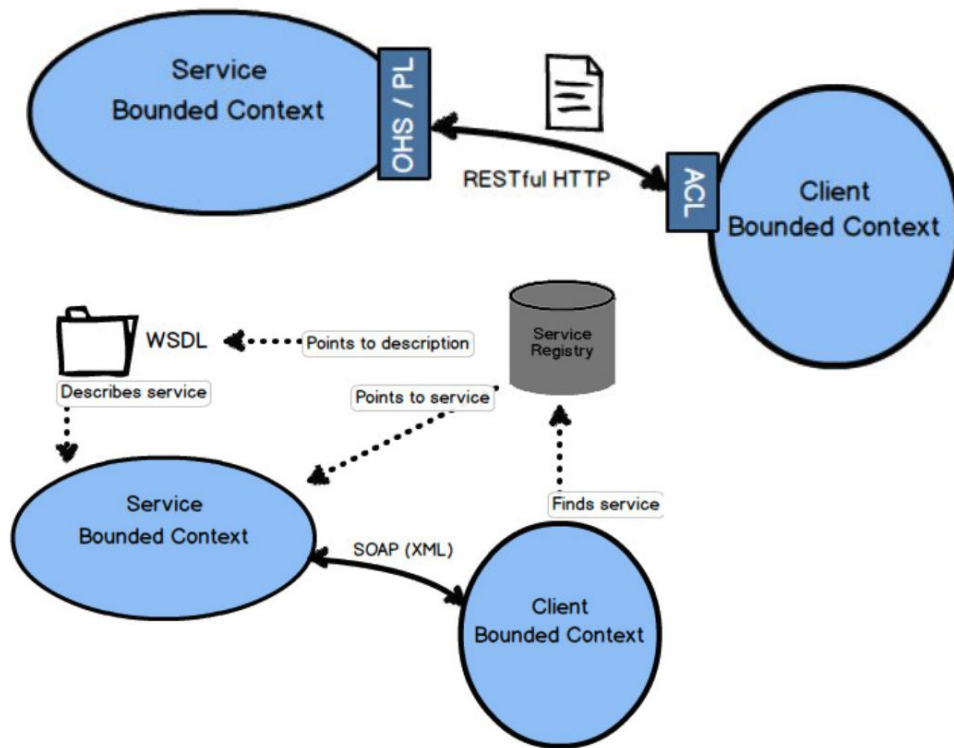


Context Mapping: Otros

- Partnership: Dos equipos responsables trabajan en conjunto sobre objetivos comunes, trabajando con sincronización constante.
- Shared Kernel: Relación cuando dos dominios comparten uno o más modelos y ambos son responsables de su desarrollo.
- Separate Ways: Situaciones donde no amerita una relación entre los dominios, más allá que compartan entidades o partes de estas.



Context Mapping: Los más comunes



Entities



- Un entidad en DDD es un objeto que tiene una entidad única y duradera dentro del modelo. Persiste a la ocurrencia de un evento.
- Comparando con OOD, una entidad es una clase, con atributos y acciones asociadas. Cada instancia de la clase dará una identidad única a esa instancia.
- En nuestro ejemplo del Chat, una entidad es el usuario. En las siguientes iteraciones podrían aparecer otras.

Resumiendo

<https://blog.bytebytego.com/p/a-crash-course-on-domain-driven-design>

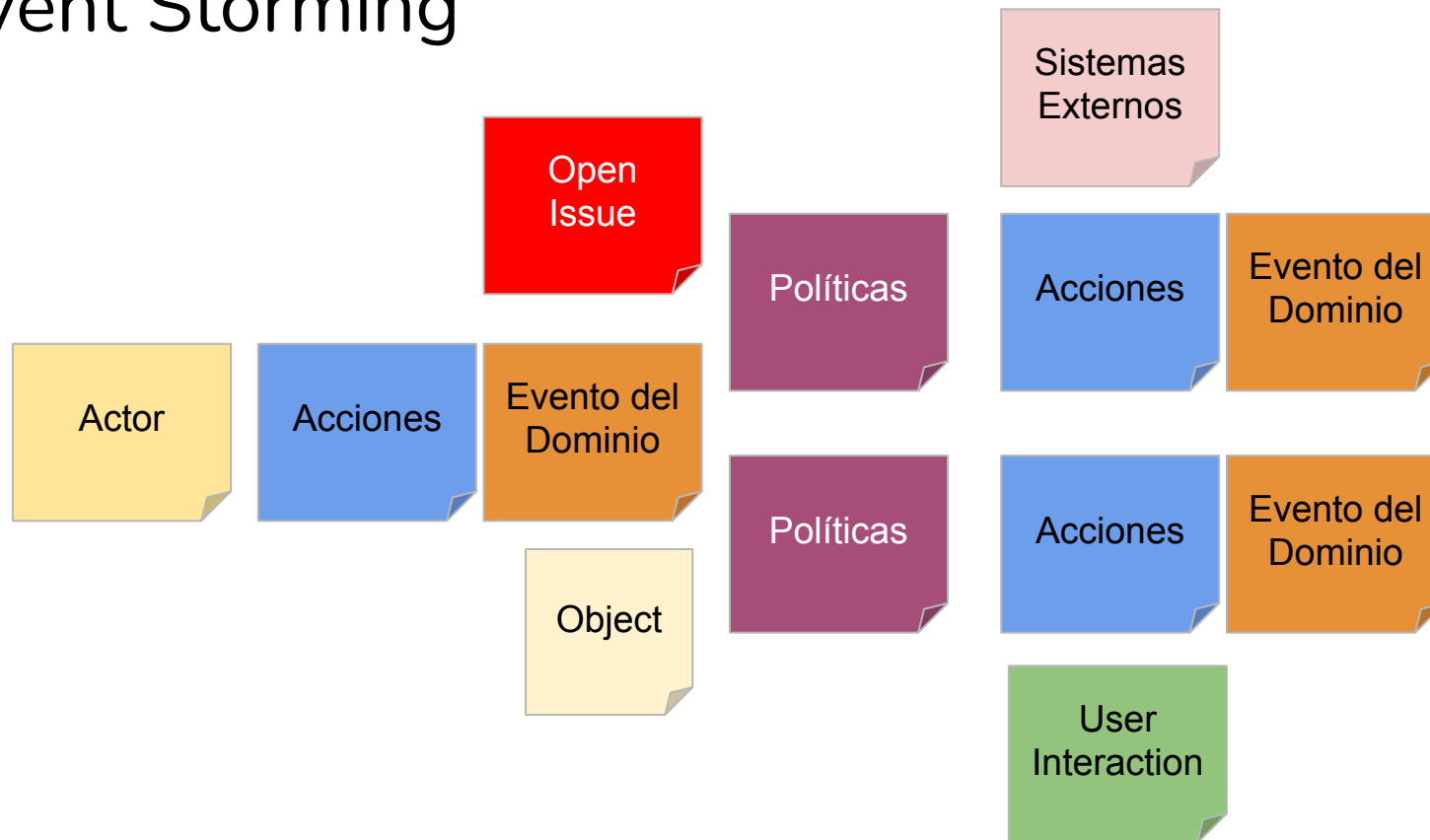
EventStorming

EventStorming es una técnica de modelado de dominio rápida, interactiva y participativa. Esta estrategia fue introducida por Alberto Brandolini en 2013. Se utiliza para llevar a los equipos de desarrollo y negocios a una profunda comprensión de un dominio o problema específico.

Se trabaja en equipo y queda como documentación entre el usuario final, los arquitectos y desarrolladores.

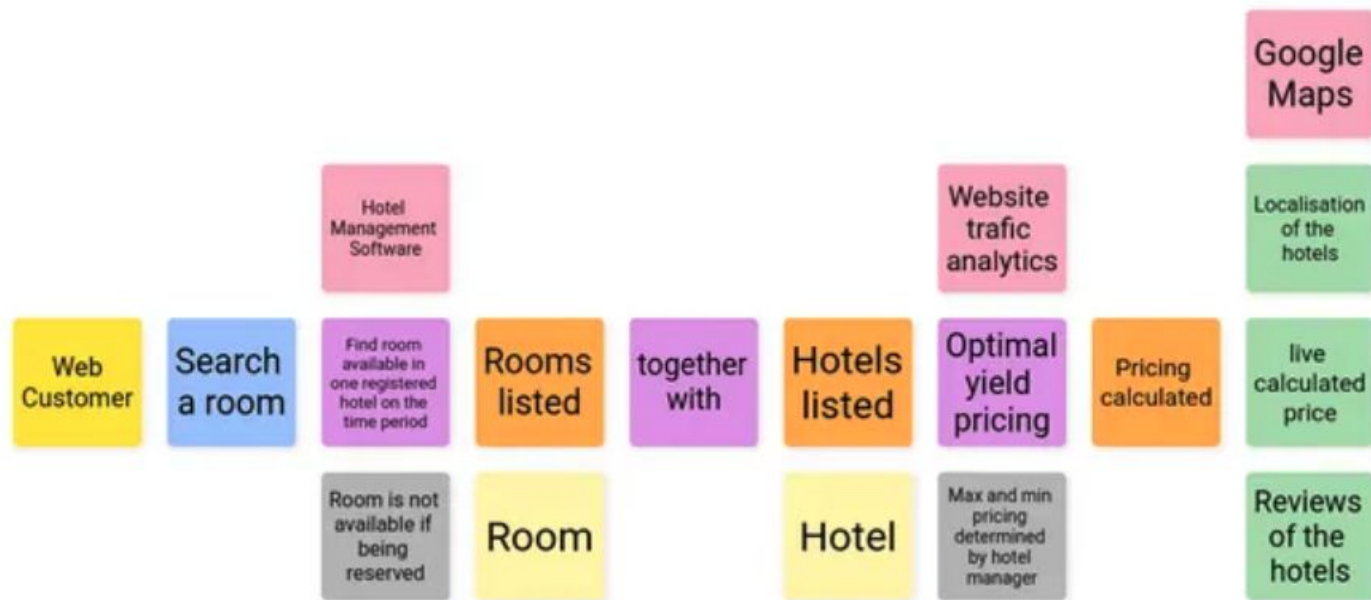
Identifica Eventos en el dominio que son significativos para el negocio, y el impacto de estos eventos a través del dominio.

Event Storming



Eventstorming: Ejemplo

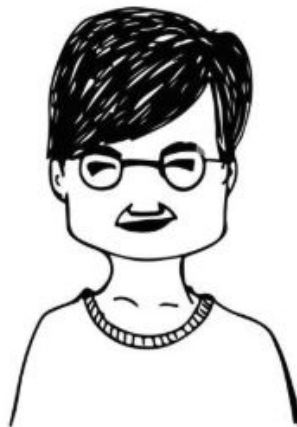
Booking process



Ejemplo

Conversación acerca de un proceso...

Se reúnen los gerentes de Ventas, Logística, Operaciones y Facturación para diseñar un sistema.



Requerimientos

Cada uno cuenta lo que el sistema debe hacer, desde su punto de vista:

- **Gte. Ventas:**
 - Cuando el usuario presiona el botón de comprar, se genera un pedido con los ítems que se necesitan y la cantidad.
- **Gte Operaciones:**
 - Cuando se recibe un pedido, se verifica si todos los ítems solicitados están disponibles y se lo aprueba, reservando los ítems en el inventario.
- **Gte. Facturación:**
 - Los pedidos aprobados se facturan y se manda a cobrar por tarjeta. El sistema de cobros tarda, así que cuando está listo, se pone el pedido como pagado, y se le pasa a Logística.
- **Gte Logística:**
 - Los pedidos pagados se programan para ser enviados, y se despachan. Una vez que se confirma la recepción, se le informa a ventas.

Requerimientos

Qué pasa si algo falla?

- Gte Operaciones:
 - Si no hay suficiente inventario se rechaza el pedido.
 - Sí facturación rechaza un pedido se libera el inventario.
 - Sí Logística devuelve un pedido, se actualiza el inventario.
- Gte. Facturación:
 - Si el cobro falla, se rechaza el pedido, informando a operaciones.
- Gte Logística:
 - Si no se puede entregar, se devuelve el operaciones los ítems, se informa a facturación y a ventas.
- Gte. Facturación:
 - Si el envío falla, asigna un crédito por lo pagado.